Global data flow analysis

- In order to efficiently optimize the code, compiler collects all the information about the program and distributes this information to each block of the flow graph. This process is known as data-flow graph analysis.
- Certain optimization can only be achieved by examining the entire program. It can't be achieve by examining just a portion of the program.
- Here using the value of the variable, we try to find out that which definition of a variable is applicable in a statement.

Based on the local information a compiler can perform some optimizations. For example, consider the following code:

```
x = a + b;
  x = 6 * 3
```

- In this code, the first assignment of **x** is useless. The value computer for **x** is never used in the program.
- At compile time the expression **6*3** will be computed, simplifying the second assignment statement to **x = 18**;

Some optimization needs more global information. For example, consider the following code:

```
a = 1;
  b = 2;
  c = 3;
  if (....) x = a + 5;
  else x = b + 4;
  c = x + 1;
```

In this code, at **line 3** the initial assignment is useless and **x +1** expression can be simplified as **7**.

But it is less obvious that how a compiler can discover these facts by looking only at one or two consecutive statements. A more global analysis is required so that the compiler knows the following things at each point in the program:

- Which variables are guaranteed to have constant values
- Which variables will be used before being redefined

Data flow analysis is used to discover this kind of property. The data flow analysis can be performed on the program's control flow graph.

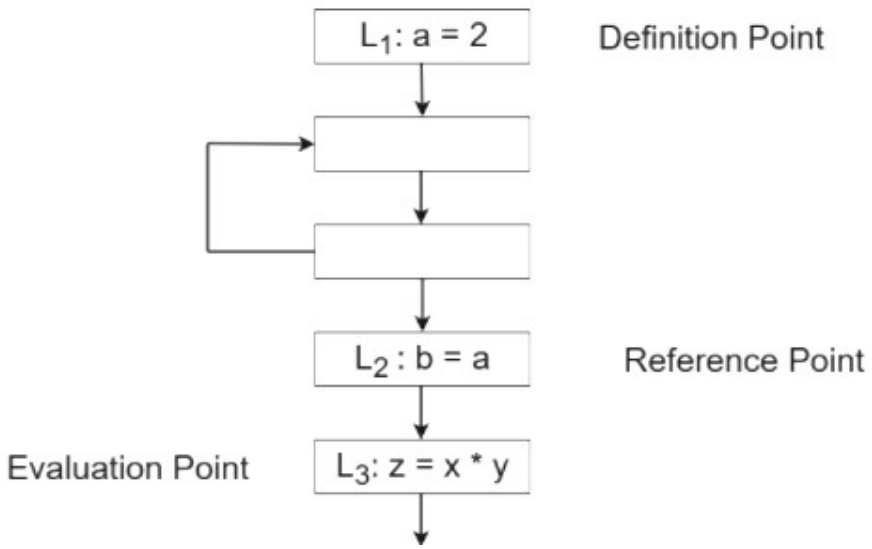
Data flow analysis in Compiler

It is the analysis of flow of data in control flow graph, i.e., the analysis that determines the information regarding the definition and use of data in program. With the help of this analysis optimization can be done. In general, it is the process in which values are computed using data flow analysis. The data flow property represents information which can be used for optimization.

Basic Terminologies:

- **Definition Point:** a point in a program containing some definition.
- **Reference Point:** a point in a program containing a reference to a data item.
- **Evaluation Point:** a point in a program containing evaluation of expression.

Example:

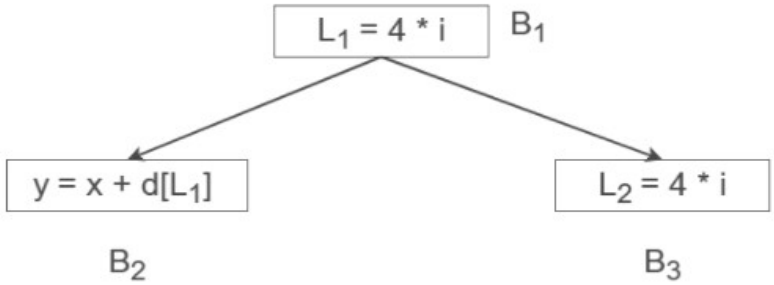


Data Flow Properties:

1. **Available Expression**
2. **Reaching Definition**
3. **Live variable**
4. **Busy Expression**

1. Available Expression: An expression is said to be available at a program point '**p**' iff along paths it is reaching to **p**.
An Expression is available at its evaluation point.
An expression **a+b** is said to be available if none of the operands gets modified before their use.

Example:

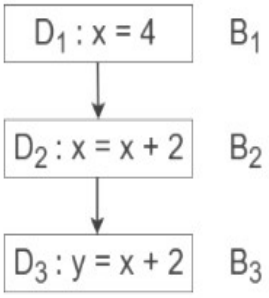


Expression 4 * i is available for block B_2 , B_3

- **Advantage:**
  It is used to eliminate common sub expressions.

2. Reaching Definition: A definition **D** is reaches a point '**p**' if there is path from **D** to **p** in which **D** is not killed, i.e., not redefined.

Example:

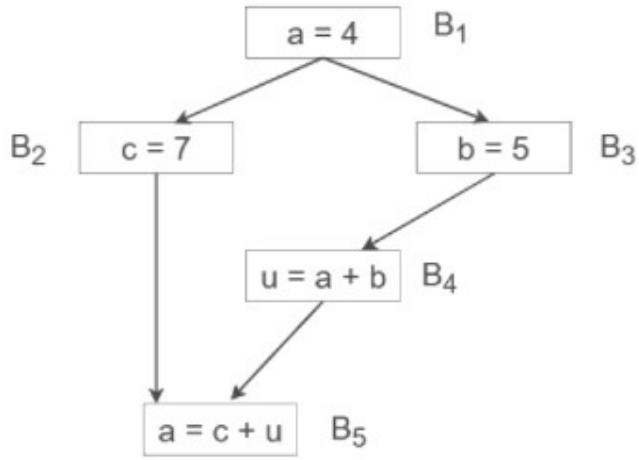


D_1 is reaching definition for B_2 but not for B_3 since it is killed by D_2

- **Advantage –**
  It is used in constant and variable propagation.

3. Live variable: A variable is said to be live at some point **p** if the variable is used before it is redefined from **p** to end. Else it becomes dead.

Example:



a is live at block B_1 , B_3 , B_4 but killed at B_5

- **Advantage –**
  1. It is useful for register allocation.
  2. It is used in dead code elimination.

4. Busy Expression: An expression is busy along a path iff its evaluation exists along that path and none of its operand definition exists before its evaluation along the path.

Advantage –
It is used for performing code movement optimization.

Data-flow information can be collected by setting up and solving systems of equations of the form :

$$\text{out [S]} = \text{gen [S]} \cup (\text{in [S]} - \text{kill [S]})$$

This equation can be read as "the information at the end of a statement is either generated within the statement, or enters at the beginning and is not killed as control flows through the statement." Such equations are called data-flow equation.

Loops In Flow Graph

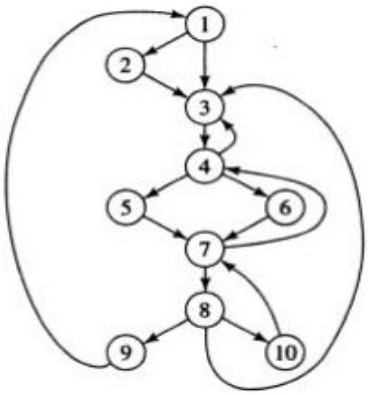
A graph representation of three-address statements, called a flow graph, is useful for understanding the code, even if the graph is not explicitly constructed by a code-generation algorithm. Nodes in the flow graph represent computations, and the edges represent the flow of control.

Dominators:

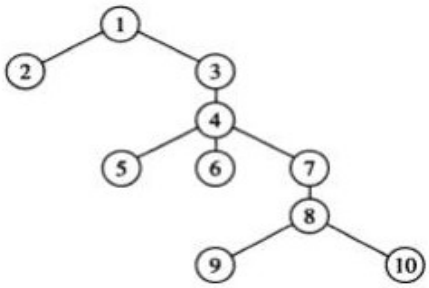
In a flow graph, a node **d** dominates node **n**, if every path from initial node of the flow graph to **n** goes through **d**. This will be denoted by **d dom n**. The initial node dominates all the remaining nodes in the flow graph and the entry of a loop dominates all nodes in the loop. Similarly every node dominates itself.

Example:

*In the flow graph below, Initial node, node1 dominates every node. *node 2 dominates itself
*node 3 dominates all but 1 and 2. *node 4 dominates all but 1,2 and 3.
*node 5 and 6 dominates only themselves, since flow of control can skip around either by goin through the other.
*node 7 dominates 7,8 ,9 and 10. *node 8 dominates 8,9 and 10.
*node 9 and 10 dominates only themselves.



(Flow graph)

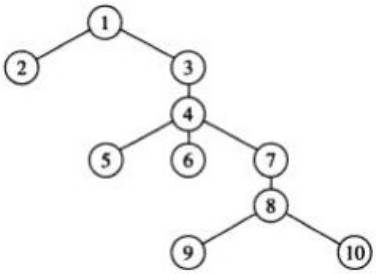


(Dominator tree)

The way of presenting dominator information is in a tree, called the **dominator tree**, in which

- The initial node is the root.
- The parent of each other node is its immediate dominator.
- Each node **d** dominates only its descendents in the tree.

The existence of dominator tree follows from a property of dominators; each node has a unique immediate dominator in that is the last dominator of **n** on any path from the initial node to **n**.



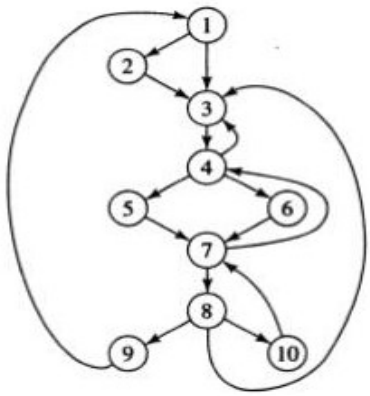
D(1)={1}
D(2)={1,2}
D(3)={1,3}
D(4)={1,3,4}
D(5)={1,3,4,5}
D(6)={1,3,4,6}
D(7)={1,3,4,7}
D(8)={1,3,4,7,8}
D(9)={1,3,4,7,8,9}
D(10)={1,3,4,7,8,10}

Natural Loops:

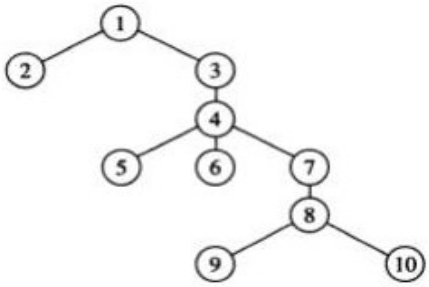
One application of dominator information is in determining the loops of a flow graph suitable for improvement. There are two essential properties of loops:

Ø A loop must have a single entrypoint, called the header. This entry point-dominates all nodes in the loop, or it would not be the sole entry to the loop.
Ø There must be at least one way to iterate the loop(i.e.)at least one path back to the header. One way to find all the loops in a flow graph is to search for edges in the flow graph whose heads dominate their tails. If **a→b** is an edge, b is the head and a is the tail. These types of edges are called as back edges.

Example:



(Flow graph)

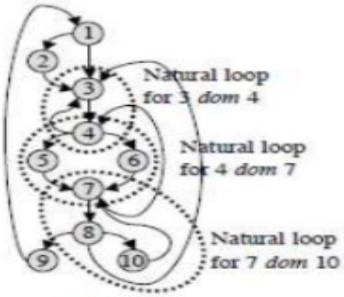


(Dominator tree)

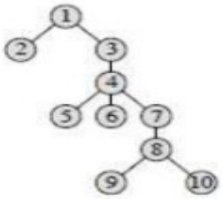
In the above graph,

7→4 is back edge because 4 DOM 7
10 →7 is back edge because 7 DOM 10
4→3 is back edge because 3 DOM 4
8→3 is back edge because 3 DOM 8
9 →1 is back edge because 1 DOM 9

The above edges will form loop in flow graph. Given a back edge **n → d**, we define the natural loop of the edge to be d plus the set of nodes that can reach n without going through d. Node d is the header of the loop.



CFG

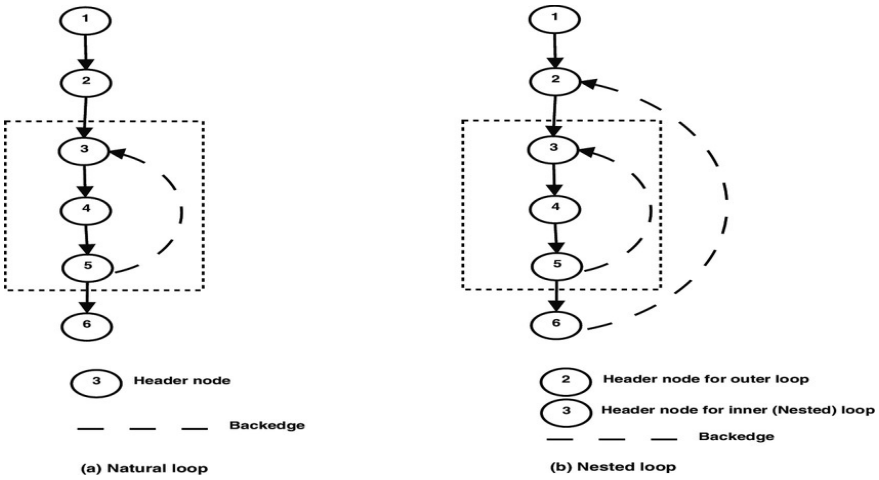


Dominator tree

Inner loops:

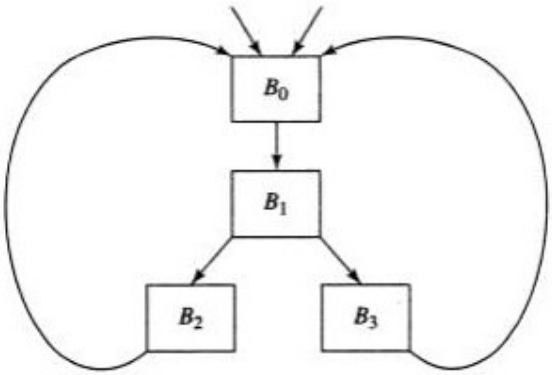
If we use the natural loops as “the loops”, then we have the useful property that unless two loops have the same header, they are either disjointed or one is entirely contained in the other. Thus, neglecting loops with the same header for the moment, we have a natural notion of inner loop: one that contains no other loop.

When two natural loops have the same header, but neither is nested within the other, they are combined and treated as a single loop.

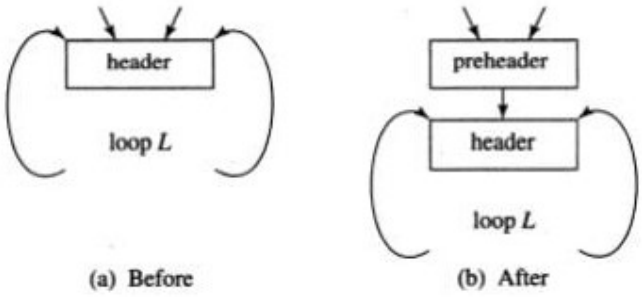


Pre-Headers:

Several transformations require us to move statements “before the header”. Therefore begin treatment of a loop L by creating a new block, called the preheader. The pre-header has only the header as successor, and all edges which formerly entered the header of L from outside L instead enter the pre-header. Edges from inside loop L to the header are not changed. Initially the pre-header is empty, but transformations on L may place statements in it.



(Two loops with same header)



(Introducing pre-header)

Reducible flow graphs:

Reducible flow graphs are special flow graphs, for which several code optimization transformations are especially easy to perform, loops are unambiguously defined, dominators can be easily calculated, data flow analysis problems can also be solved efficiently.

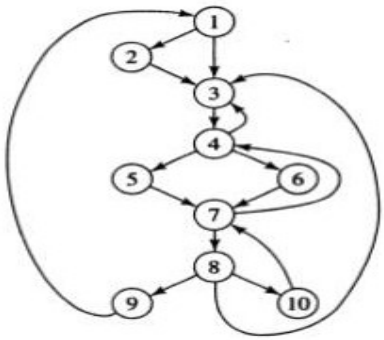
Definition:

A flow graph G is reducible if and only if we can partition the edges into two disjoint groups, forward edges and back edges, with the following properties.

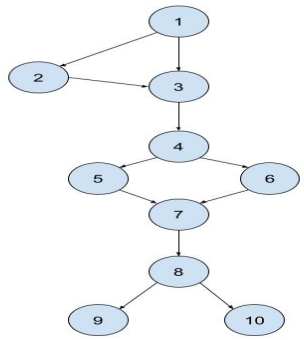
1. The forward edges from an acyclic graph in which every node can be reached from initial node of G.

2. The back edges consist only of edges where heads dominate their tails.

Example: The above flow graph is reducible. If we know the relation DOM for a flow graph, we can find and remove all the back edges. The remaining edges are forward edges. If the forward edges form an acyclic graph, then we can say the flow graph is reducible. In the above example if we remove the five back edges 4→3, 7→4, 8→3, 9→1 and 10→7 whose heads dominate their tails, the remaining graph is acyclic.



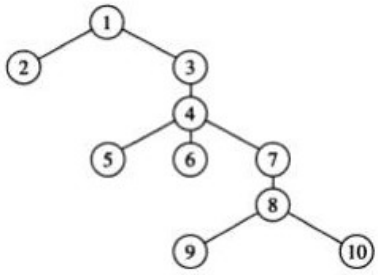
After removing forward edges:



This forms a DAG. Hence it is reducible.

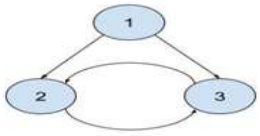
How to get Reduced flow graphs

- Remove the back edges (does not contain a loop)
- We repeatedly delete edges and collapse together pairs of nodes (x, y), where x is the only predecessor of y.

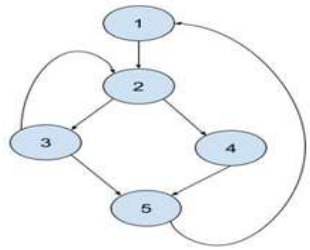


We can collapse (8,9), (8,10), (7,8), (4,5), (4,6), (4,7), (3,4), (1,2) and (1,3). The reduced flow graph will be:

Some more examples:



(Non-reducible Flow Graph)



(Reducible Flow Graph)

<u>Optimization can be categorized broadly into two types : machine independent and machine dependent.</u>

Machine-independent Optimization

In this optimization, the compiler takes an intermediate code and transforms a part of the code that does not involve any CPU registers and/or absolute memory locations. For example:

```
do
{
   item = 10;
   value = value + item;
} while(value<100);
```

This code involves repeated assignment of the identifier item, which if we put this way:

```
Item = 10;
do
{
   value = value + item;
} while(value<100);
```

It will not only save the CPU cycles, but also can be used on any processor.

Loop unrolling and Loop fusions are two loop optimization techniques used as loop transformation technique that helps to optimize the execution time of a program. We basically remove or reduce iterations. These increases the program's speed by eliminating loop control instruction and loop test instructions.

<u>Loop unrolling</u>

```
// This program does not uses loop unrolling.
#include<stdio.h>

int main(void)
{
    for (int i=0; i<5; i++)
        printf("Hello\n"); //print hello 5 times

    return 0;
}
```

```
// This program uses loop unrolling.
#include<stdio.h>

int main(void)
{
    // unrolled the for loop in program 1
    printf("Hello\n");
    printf("Hello\n");
    printf("Hello\n");
    printf("Hello\n");
    printf("Hello\n");

    return 0;
}
```

Loop fusions

// this program does not use loop fusion

```
for (i=0;i<=n;i++)
{
        a[i] = a[i]+10;
}
for (j=0;j<=n;j++)
{
        b[j] = b[j]-10;
}
```

// this program uses loop fusion

```
for (i=0;i<=n;i++)
{
        a[i] = a[i]+10;
        b[i] = b[i]-10;
}
```

Machine-dependent Optimization

Machine-dependent optimization is done after the target code has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine-dependent optimizers put efforts to take maximum advantage of memory hierarchy.